Algorithm Specification

Sambridda Ranabhat

Water Automation Project for Mechatronics Engineering

SC1 (Sump Station)

CONSTANTS

```
ADDR_SC1            = 0x01
ADDR_MASTER         = 0x10
CMD_PING            = 0x01
CMD_ACK             = 0x02
CMD_REPORT          = 0x31
BOOT_DELAY          = 60s
LEVEL_DELTA         = 5%
TANK_DEPTH_CM       = 300cm
BACKOFF MS          = 521ms
MAX_RETRIES         = 3
lastTempRead        = 0
TEMP_READ_INTERVAL  = 600000  // 10 minutes in ms
TEMP_SETTLE_TIME    = 2000    // 2 second DHT11 settle window
tempSettleStart     = 0
waitingForDHT       = FALSE
lastUltrasonicRead  = 0
ULTRASONIC_INTERVAL = 5000    // 5 seconds in ms
currentLevel        = 0
currentTemp         = 25.0    // default temperature
TEMP_MAX_WAIT       = 5000    // force read after 5 seconds even if HC12 is
busy
```

Standard WACP v4.0 addresses and command codes. `LEVEL_DELTA` is the minimum change in water level before SC1 proactively sends a report.

BOOT

```
ON POWER UP:
    safe_state()
    wait(60s)
    currentLevel = readUltrasonic()
    lastReportedLevel = currentLevel
    lastTransmitTime = now()
    retryCount        = 0
```

```
    GOTO IDLE_MONITOR
```

60 second delay allows sensors and capacitors to stabilize before any logic runs. Initial level is recorded as baseline.

IDLE_MONITOR

```
LOOP:

// --- TEMPERATURE: Stage 1 -- trigger settle ---
IF NOT waitingForDHT AND (now() - lastTempRead) >= TEMP_READ_INTERVAL:
    IF NOT HC12.available():
        waitingForDHT  = TRUE
        tempSettleStart = now()

// --- TEMPERATURE: Stage 2 -- read after settle ---
IF waitingForDHT AND (now() - tempSettleStart) >= TEMP_SETTLE_TIME:
    IF NOT HC12.available():        // only read when radio is quiet
        readTemperature()
        lastTempRead = now()
        waitingForDHT = FALSE

    // if HC12 is busy, stay in waitingForDHT and retry next loop

    // NEW -- called every 5 seconds
IF (now() - lastUltrasonicRead) >= ULTRASONIC_INTERVAL:
    currentLevel = readUltrasonic()
    lastUltrasonicRead = now()


    IF radio.packetAvailable():
        packet = radio.receive()
        IF validateChecksum(packet) AND packet.DEST == ADDR_SC1:
            IF packet.CMD == CMD_PING:
                sendACK(CMD_PING)
                GOTO REPORT_GENERATION
        ELSE:
            discard packet

    IF abs(currentLevel - lastReportedLevel) >= LEVEL_DELTA:
        GOTO REPORT_GENERATION
```

SC1 spends most of its life here. Two exit conditions only — a PING from Master, or a significant level change. ACK is sent immediately on PING before anything else.

REPORT_GENERATION

```
    PRE_H = 0xAA
    PRE_L = 0x55
    DEST  = ADDR_MASTER
    CMD   = CMD_REPORT        // 0x31
    VAL   = currentLevel      // 0-100%
    CS    = DEST XOR CMD XOR VAL
    GOTO REPORT_QUEUED
```

Builds the 6-byte WACP packet. A fresh ultrasonic reading is taken here to ensure the data being sent is current.

REPORT_QUEUED

```
enqueue(packet)
    GOTO REPORT_SENT
```

Packet enters the transmission queue. This buffer prevents logic crashes if multiple events fire simultaneously (Anti-Collision Protocol).

REPORT_SENT

```
REPORT_SENT
    IF NOT radio.available():    // only transmit if channel is clear
        radio.transmit(packet)
        lastTransmitTime = now()
        lastReportedLevel = currentLevel
        retryCount = 0
    GOTO WAITING_FOR_ACK
```

Packet is transmitted. Baseline level is updated so future delta checks are relative to what was last reported.

WAITING_FOR_ACK

```
startWait = now()
   LOOP:
       IF radio.packetAvailable():
           ack = radio.receive()
           IF validateChecksum(ack) AND ack.DEST == ADDR_SC1:
               IF ack.CMD == CMD_ACK AND ack.VAL == CMD_REPORT:
                   retryCount = 0
                   GOTO IDLE_MONITOR

       IF (now() - startWait) > BACKOFF_MS:
           retryCount++
           IF retryCount < MAX_RETRIES:
               GOTO REPORT_SENT
           ELSE:
               retryCount = 0
               GOTO IDLE_MONITOR
   REPEAT LOOP
```

Stop-and-Wait protocol. SC1 holds here until the Master confirms receipt. If no ACK within 1890ms, retransmits up to 3 times before giving up and returning to listen mode.

Helpers

```
FUNCTION validateChecksum(packet):
    calculated = packet.DEST XOR packet.CMD XOR packet.VAL
    RETURN (calculated == packet.CS)

FUNCTION readUltrasonic():
    // Temperature compensated speed of sound (m/s)
    speedOfSound = 331.3 + (0.606 × currentTemp)

    // Trigger pulse
    TRIG = LOW
    wait(2µs)
    TRIG = HIGH
    wait(10µs)
    TRIG = LOW

    // Read echo with timeout (30ms covers up to ~514cm)
    ESP.wdtFeed()
```

```
    duration = pulseIn(ECHO, HIGH, 30000)

    IF duration == 0:
        RETURN currentLevel    // sensor failed -- return last known value

    // Convert duration to distance in cm
    raw_cm = (duration × speedOfSound) / 20000.0

    level_pct = map(raw_cm, TANK_DEPTH_CM, 0, 0, 100)
    level_pct = constrain(level_pct, 0, 100)
    RETURN level_pct

FUNCTION sendACK(cmd_being_acked):
    PRE_H = 0xAA
    PRE_L = 0x55
    DEST  = ADDR_MASTER
    CMD   = CMD_ACK
    VAL   = cmd_being_acked
    CS    = DEST XOR CMD XOR VAL
    radio.transmit(packet)
```

`sendACK()` immediately fires back to Master without queuing — it's a direct reply.
`validateChecksum()` silently drops corrupt packets.

```
FUNCTION readTemperature():
    newTemp = dht.readTemperature()
    IF NOT isnan(newTemp):
        currentTemp = newTemp
```

SC2 (Basement Control)

Constants

```
ADDR_SC2            = 0x02
```

```
ADDR_MASTER          = 0x10
CMD_PING             = 0x01
CMD_ACK              = 0x02
CMD_REPORT           = 0x32
CMD_PUMP             = 0x20
CMD_FAULT            = 0x30
CMD_TIMER            = 0x21
CMD_AMP_P1           = 0x40
CMD_TIMER_DONE       = 0x22
CMD_AMP_P2           = 0x41
BACKOFF_MS           = 383ms
MAX_RETRIES          = 3
LEVEL_DELTA          = 5%
TANK_DEPTH_CM        = <physical measurement>
TANK_LOW             = 25%
TANK_HIGH            = 80%
DRY_RUN_THRESHOLD    = 0.5A
OVERLOAD_THRESHOLD   = 8.0A
DRY_RUN_RETRY_DELAY = 10s
MAX_DRY_RETRIES      = 3
TEMP_INTERVAL        = 600s
DISCONNECT_PERIOD    = 90s
QUEUE_SIZE           = 8
TEMP_SETTLE_TIME     = 2000
TEMP_MAX_WAIT        = 5000    // force read after 5 seconds even if
HC12 is busy
```

Pump State Structure

```
STRUCT PumpState:
    isOn          = FALSE
    isFault       = FALSE
    isLocked      = FALSE
```

```
    faultCode      = 0
    dryRunCount    = 0

pumps[2] = {PumpState, PumpState}   // index 0 = P1, index 1 = P2
```

Packet Queue Structure

```
STRUCT QueuedPacket:
    data[6]        // 6 byte WACP packet
    priority       // 1=critical, 2=important, 3=normal, 4=low
    retries        // current retry count
    waitingAck     // TRUE if waiting for ACK

packetQueue[QUEUE_SIZE]
queueCount     = 0
lastSendTime   = 0
```

State Variables

```
currentTemp       = 25.0       // default temperature for calibration
lastTempRead      = 0          // timestamp of last DHT11 read
lastReportedLevel = 0          // last level sent to Master
timerActive       = FALSE      // timer running flag
timerEndMillis    = 0          // timer expiry timestamp
lastMasterHeard   = 0          // watchdog timestamp
p2Frozen          = FALSE      // P2 frozen due to disconnect
inPacket[6]                    // incoming packet buffer
outPacket[6]                   // outgoing packet buffer
lastTempRead       = 0
TEMP_READ_INTERVAL = 600000    // 10 minutes in ms
TEMP_SETTLE_TIME   = 2000      // 2 second DHT11 settle window
tempSettleStart    = 0
waitingForDHT      = FALSE
dryRunRetryPending = FALSE
dryRunRetryTime    = 0
dryRunRetryPumpID  = 0
lastUltrasonicRead  = 0
```

```
ULTRASONIC_INTERVAL = 5000      // 5 seconds in ms
currentLevel        = 0
```

BOOT

```
ON POWER UP:
    RELAY1 = OFF                // P1 safe state
    RELAY2 = OFF                // P2 safe state
    MUX_SEL = unset             // only used when pump is running
    pumps[0] = {FALSE, FALSE, FALSE, 0, 0}
    pumps[1] = {FALSE, FALSE, FALSE, 0, 0}
    timerActive   = FALSE
    timerEnd      = 0
    p2Frozen      = FALSE
    HC12.begin(9600)
    dht.begin()
    lastReportedLevel = readUltrasonic()
    GOTO IDLE_MONITOR
```

No boot delay — SC2 handles no electrical appliances directly on startup. Relays default to LOW (safe). Multiplexer left uninitialised until a pump is running.

IDLE_MONITOR

```
LOOP:
    // --- TEMPERATURE: Stage 1 -- trigger settle ---
IF NOT waitingForDHT AND (now() - lastTempRead) >=
TEMP_READ_INTERVAL:
    IF NOT HC12.available():
        waitingForDHT  = TRUE
        tempSettleStart = now()

// --- TEMPERATURE: Stage 2 -- read after settle ---
IF waitingForDHT AND (now() - tempSettleStart) >= TEMP_SETTLE_TIME:
    IF NOT HC12.available():        // only read when radio is quiet
        readTemperature()
```

```
        lastTempRead = now()
        waitingForDHT = FALSE
    // if HC12 is busy, stay in waitingForDHT and retry next loop

   // NEW -- called every 5 seconds
IF (now() - lastUltrasonicRead) >= ULTRASONIC_INTERVAL:
    currentLevel = readUltrasonic()
    lastUltrasonicRead = now()


    // --- TRIGGER 1: Timer expiry ---
IF timerActive AND now() >= timerEndMillis:
    timerActive = FALSE
    RELAY2 = OFF
    pumps[1].isOn = FALSE
    pushToQueue(ADDR_MASTER, CMD_PUMP, 2, PRIORITY_IMPORTANT)
    pushToQueue(ADDR_MASTER, CMD_TIMER_DONE, 0, PRIORITY_IMPORTANT)

    // --- TRIGGER 2: Incoming packet from Master ---
    IF HC12.available() >= 6:
        readPacket(currentLevel)

    // --- TRIGGER 3: Async level change ---
    IF abs(currentLevel - lastReportedLevel) >= LEVEL_DELTA:
        pushToQueue(ADDR_MASTER, CMD_REPORT, currentLevel,
PRIORITY_NORMAL)
        lastReportedLevel = currentLevel

    // --- TRIGGER 4: Current monitoring if pump is ON ---
    IF pumps[0].isOn OR pumps[1].isOn:
        monitorCurrentSensor()

    // --- TRIGGER 5: Master disconnect watchdog ---
    checkMasterConnection()

    // --- Process outgoing queue ---
    processQueue()

// --- TRIGGER: Dry run retry ---
```

```
IF dryRunRetryPending AND (now() >= dryRunRetryTime):
    dryRunRetryPending = FALSE
    pumpID = dryRunRetryPumpID
    IF NOT pumps[pumpID].isLocked AND NOT pumps[pumpID].isFault:
        relay = (pumpID == 0) ? RELAY1 : RELAY2
        relay = ON
        pumps[pumpID].isOn = TRUE


    REPEAT LOOP
```

READ_TEMPERATURE

```
FUNCTION readTemperature():
    newTemp = dht.readTemperature()
    IF NOT isnan(newTemp):
        currentTemp = newTemp
```

READ_ULTRASONIC

```
FUNCTION readUltrasonic():
    speedOfSound = 331.3 + (0.606 × currentTemp)

    TRIG = LOW
    wait(2µs)
    TRIG = HIGH
    wait(10µs)
    TRIG = LOW

    duration = pulseIn(ECHO, HIGH, 30000)  // 30ms timeout (covers up
to ~500cm)

IF duration == 0:
    RETURN currentLevel   // sensor failed -- return last known level

    distance = (duration × speedOfSound) / 20000.0
```

```
    level    = map(distance, TANK_DEPTH_CM, 0, 0, 100)
    level    = constrain(level, 0, 100)

    RETURN level as uint8_t
```

Temperature calibration applied every read using last stored `currentTemp`. Speed of sound formula accounts for ambient temperature variation.

READ_CURRENT_SENSOR

```
FUNCTION readCurrentSensor(pumpID):
    // Select correct ACS712 via multiplexer
    IF pumpID == 0:
        MUX_SEL = LOW       // reads CS1 = P1 current
    ELSE:
        MUX_SEL = HIGH      // reads CS2 = P2 current

    wait(100µs)             // multiplexer settling time

    raw     = analogRead(MUX_OUT)
    voltage = (raw / 1023.0) × 3.3
    amps    = abs((voltage - 1.65) / 0.066)

    RETURN amps
```

Multiplexer is only activated during this function. MUX_SEL is not set at boot or during idle — only when a pump is actively running and current reading is requested.

MONITOR_CURRENT_SENSOR

```
FUNCTION monitorCurrentSensor():
    IF pumps[0].isOn:
        amps = readCurrentSensor(0)
```

```
        IF amps < DRY_RUN_THRESHOLD:
            GOTO handleDryRun(0, amps)
            RETURN
        IF amps > OVERLOAD_THRESHOLD:
            GOTO handleOverload(0, amps)
            RETURN

    IF pumps[1].isOn:
        amps = readCurrentSensor(1)
        IF amps < DRY_RUN_THRESHOLD:
            GOTO handleDryRun(1, amps)
            RETURN
        IF amps > OVERLOAD_THRESHOLD:
            GOTO handleOverload(1, amps)
            RETURN
```

HANDLE_DRY_RUN

```
FUNCTION handleDryRun(pumpID, amps):
    relay = (pumpID == 0) ? RELAY1 : RELAY2
    relay = OFF
    pumps[pumpID].isOn = FALSE
    pumps[pumpID].dryRunCount++

    IF pumps[pumpID].dryRunCount < MAX_DRY_RETRIES:
        // Schedule retry -- don't block
        dryRunRetryPending  = TRUE
        dryRunRetryTime     = now() + 10000    // 10 seconds from now
        dryRunRetryPumpID   = pumpID
        RETURN

    // Max retries reached
    pumps[pumpID].dryRunCount = 0
    faultCode = (pumpID == 0) ? 1 : 3
    GOTO triggerEmergencyStop(pumpID, faultCode, amps)
```

3 retries at 10 second intervals. Counter resets to 0 after lockout so a manual restart from

Master gives a fresh 3 attempts.

HANDLE_OVERLOAD

```
FUNCTION handleOverload(pumpID, amps):
    // No retries -- immediate lockout of both pumps
    RELAY1 = OFF
    RELAY2 = OFF
    pumps[0].isOn = FALSE
    pumps[1].isOn = FALSE

    faultCode = (pumpID == 0) ? 2 : 4  // 2=P1 overload, 4=P2 overload
    GOTO triggerEmergencyStop(pumpID, faultCode, amps)
```

TRIGGER_EMERGENCY_STOP

```
FUNCTION triggerEmergencyStop(pumpID, faultCode, amps):
    RELAY1 = OFF
    RELAY2 = OFF
    pumps[0].isOn    = FALSE
    pumps[1].isOn    = FALSE
    timerActive      = FALSE
    pumps[pumpID].isFault  = TRUE
    pumps[pumpID].isLocked = TRUE

    // Push fault packet at CRITICAL priority
    pushToQueue(ADDR_MASTER, CMD_FAULT, faultCode, PRIORITY_CRITICAL)

    // Push amperage packet at CRITICAL priority immediately after
    cmd = (pumpID == 0) ? CMD_AMP_P1 : CMD_AMP_P2
    val = (uint8_t)(amps × 10)          // e.g. 7.8A → 78
    pushToQueue(ADDR_MASTER, cmd, val, PRIORITY_CRITICAL)
```

Two packets pushed back to back at critical priority. The master will not ACK until both are received. Timer cancelled on emergency stop — no point counting down with pumps locked.

EXECUTE_PUMP_COMMAND

```
FUNCTION executePumpCommand(val):

    IF val == 1:                        // P1 ON
        IF NOT pumps[0].isLocked AND NOT pumps[0].isFault:
            RELAY1 = ON
            pumps[0].isOn        = TRUE
            pumps[0].dryRunCount = 0
            pushToQueue(ADDR_MASTER, CMD_PUMP, 1, PRIORITY_IMPORTANT)

    IF val == 0:                        // P1 OFF
        RELAY1 = OFF
        pumps[0].isOn        = FALSE
        pumps[0].dryRunCount = 0
        pushToQueue(ADDR_MASTER, CMD_PUMP, 0, PRIORITY_IMPORTANT)

    IF val == 3:                        // P2 ON
        IF NOT pumps[1].isLocked AND NOT pumps[1].isFault AND NOT
p2Frozen:
            currentLevel = readUltrasonic()
            IF currentLevel <= TANK_LOW:
                // Reject -- reservoir too low
                pushToQueue(ADDR_MASTER, CMD_PUMP, 2,
PRIORITY_IMPORTANT)
                RETURN
            RELAY2 = ON
            pumps[1].isOn        = TRUE
            pumps[1].dryRunCount = 0
            pushToQueue(ADDR_MASTER, CMD_PUMP, 3, PRIORITY_IMPORTANT)

    IF val == 2:                        // P2 OFF
        RELAY2 = OFF
        pumps[1].isOn        = FALSE
        pumps[1].dryRunCount = 0
        timerActive          = FALSE    // cancel timer on manual off
        pushToQueue(ADDR_MASTER, CMD_PUMP, 2, PRIORITY_IMPORTANT)
```

P2 ON is rejected silently if the reservoir is too low, pump is locked, or P2 is frozen due to Master disconnect. A pump state report is still sent so Master UI stays accurate.

SET_TIMER

```
FUNCTION setTimer(minutes):
    timerActive    = TRUE
    timerEndMillis = now() + (minutes × 60000)
```

READ_PACKET

```
FUNCTION readPacket(currentLevel):
    FOR i = 0 to 5:
        inPacket[i] = HC12.read()

    // Validate preamble
    IF inPacket[0] != 0xAA OR inPacket[1] != 0x55:
        flush HC12 buffer
        RETURN

    // Validate destination
    IF inPacket[2] != ADDR_SC2:
        flush HC12 buffer
        RETURN

    // Validate checksum
    IF NOT validateChecksum(inPacket):
        flush HC12 buffer
        RETURN

    processIncoming(currentLevel)
```

PROCESS_INCOMING

```
FUNCTION processIncoming(currentLevel):
    lastMasterHeard = now() // reset disconnect watchdog
    cmd = inPacket[3]
```

```
    val = inPacket[4]

// Only unfreeze P2 on a real command, not an ACK

    IF cmd != CMD_ACK:
    p2Frozen = FALSE


    IF cmd == CMD_PING:
        pushToQueue(ADDR_MASTER, CMD_ACK, CMD_PING, PRIORITY_LOW)
        pushToQueue(ADDR_MASTER, CMD_REPORT, currentLevel,
PRIORITY_NORMAL)
        RETURN

    IF cmd == CMD_PUMP:
        pushToQueue(ADDR_MASTER, CMD_ACK, CMD_PUMP, PRIORITY_LOW)
        executePumpCommand(val)
        RETURN

    IF cmd == CMD_TIMER:
        pushToQueue(ADDR_MASTER, CMD_ACK, CMD_TIMER, PRIORITY_LOW)
        setTimer(val)
        RETURN

    IF cmd == CMD_ACK:
        handleACK(val)
        RETURN

    IF cmd == CMD_ACK:
        handleACK(val)
        RETURN
```

`lastMasterHeard` is reset on every valid incoming packet. `p2Frozen` is cleared immediately on reconnection — auto logic or manual command can then restart P2 if needed.

CHECK_MASTER_CONNECTION

```
FUNCTION checkMasterConnection():
    IF (now() - lastMasterHeard) >= DISCONNECT_PERIOD:
        IF pumps[1].isOn AND NOT p2Frozen:
            RELAY2 = OFF
            pumps[1].isOn = FALSE
            timerActive = FALSE
            p2Frozen = TRUE
        RETURN   // disconnected -- nothing more to do

    // Master is connected -- check if we are recovering from a
freeze
    IF p2Frozen:
        p2Frozen = FALSE
        pushToQueue(ADDR_MASTER, CMD_PUMP, 2, PRIORITY_IMPORTANT)
```

Freeze is silent — no packet sent to Master since it is by definition unreachable. Report is only sent on reconnection so Master UI updates automatically.

PROCESS_QUEUE

```
FUNCTION processQueue():
    IF queueCount == 0: RETURN
    qp = packetQueue[0]
    IF qp.waitingAck:
        IF (now() - lastSendTime) >= BACKOFF_MS:
            qp.retries++
            IF qp.retries >= MAX_RETRIES:
                removeFromQueue(0)
                RETURN
            IF NOT HC12.available():    // only retry if channel is
clear
                HC12.write(qp.data, 6)
                lastSendTime = now()
            RETURN
    // Send packet only if channel is clear
```

```
    IF NOT HC12.available():
        HC12.write(qp.data, 6)
        lastSendTime = now()
        qp.waitingAck = TRUE
```

HANDLE_ACK

```
FUNCTION handleACK(ackedCmd):
    srcAddr = inPacket[2]    // who sent this ACK
    FOR i = 0 to queueCount:
        IF packetQueue[i].data[3] == ackedCmd AND
           packetQueue[i].data[2] == srcAddr:
            removeFromQueue(i)
            RETURN
```

PUSH_TO_QUEUE

```
FUNCTION pushToQueue(dest, cmd, val, priority):
    IF queueCount >= QUEUE_SIZE:
        // Find lowest priority packet
        lowestIdx  = index of packet with highest priority number
        lowestPrio = its priority value

        IF priority < lowestPrio:
            removeFromQueue(lowestIdx)  // evict lowest to make room
        ELSE:
            RETURN                      // queue full, new packet not
critical enough

    // Build packet
    qp.data = [0xAA, 0x55, dest, cmd, val, dest XOR cmd XOR val]
    qp.priority   = priority
    qp.retries    = 0
    qp.waitingAck = FALSE

    // Insert at correct sorted position
    Find insertIdx where priority < packetQueue[insertIdx].priority
```

```
    Shift packets down from insertIdx
    packetQueue[insertIdx] = qp
    queueCount++
```

HELPERS

```
FUNCTION calculateChecksum(dest, cmd, val):
    RETURN dest XOR cmd XOR val

FUNCTION validateChecksum(packet):
    calculated = packet[2] XOR packet[3] XOR packet[4]
    RETURN (calculated == packet[5])

FUNCTION removeFromQueue(idx):
    Shift all packets from idx+1 downward
    queueCount--
```

Control Station (The Arbeiter)

Constants

```
ADDR_MASTER         = 0x10
ADDR_SC1            = 0x01
ADDR_SC2            = 0x02
CMD_PING            = 0x01
CMD_ACK             = 0x02
CMD_REPORT_SC1      = 0x31
CMD_REPORT_SC2      = 0x32
CMD_PUMP            = 0x20
CMD_FAULT           = 0x30
CMD_TIMER           = 0x21
CMD_TIMER_DONE      = 0x22
CMD_AMP_P1          = 0x40
CMD_AMP_P2          = 0x41
BACKOFF_MS          = 251ms
MAX_RETRIES         = 3
```

```
HEARTBEAT_INTERVAL   = 60s
SC1_POLL_OFFSET      = 1s
SC2_POLL_OFFSET      = 31s
INACTIVITY_LIMIT     = 60s
SC_TIMEOUT           = 180s
C1_LOW               = 25%
C1_HIGH              = 80%
C2_FULL              = 80%
NTP_SYNC_HOUR        = 7
TIMEZONE_OFFSET      = 19800s    // UTC+5:45 Kathmandu
QUEUE_SIZE           = 8
```

Pump State Structure

```
STRUCT PumpState:
    isOn          = FALSE
    isFault       = FALSE
    isLocked      = FALSE
    faultCode     = 0
    dryRunCount   = 0
    statusLabel   = "OFF"

pumps[2] = {PumpState, PumpState}   // index 0 = Overhead, index 1 =
Basement
```

Packet Queue Structure

```
STRUCT QueuedPacket:
    data[6]       // 6 byte WACP packet
    priority      // 1=critical, 2=important, 3=normal, 4=low
    retries       // current retry count
    waitingAck    // TRUE if waiting for ACK

packetQueue[QUEUE_SIZE]
queueCount    = 0
lastSendTime  = 0
```

State Variables

```
// Network state
c1Level             = 0          // last known reservoir level from SC1
c2Level             = 0          // last known overhead tank level from SC2
sc1Connected        = FALSE
sc2Connected        = FALSE
lastSC1Heard        = 0
lastSC2Heard        = 0

// Fault handling
activeFault         = 0
pendingFault        = 0
waitingForAmp       = FALSE
lastKnownAmps       = 0.0
previousScreen      = SCREEN_HOME

// User input
sourceActive        = FALSE
sourceToggleIndex   = 0

// Timer
timerActive         = FALSE
timerEndMillis      = 0
timerIndex          = 0
timerPresets[9]     = {30, 60, 90, 120, 150, 180, 210, 240, 270}
timerLabels[9]      = {"30 min", "1 hr", "1 hr 30 min", "2 hr",
                       "2 hr 30 min", "3 hr", "3 hr 30 min",
                       "4 hr", "4 hr 30 min"}

// UI state
currentScreen       = SCREEN_HOME
menuIndex           = 0
maxMenuItems        = 5
menuYPositions[5]   = {96, 114, 132, 149, 165}
selectedPump        = 0
pumpActionIndex     = 0
selectingAction     = FALSE
screensaverActive   = FALSE
lastActivityTime    = 0
// WiFi diagnostic
lastConnectionSuccess = FALSE
runningDiagnostic     = FALSE
```

```
// Clock
currentHourStr      = "12"
currentMinStr       = "00"
currentAmPm         = "AM"
// Timing
lastHeartbeat       = 0
nextNTPSync         = 0
// Packet buffers
inPacket[6]
outPacket[6]
lastButtonTime = 0
DEBOUNCE_INTERVAL = 200
buzzerState       = IDLE     // IDLE, BEEP_ON, BEEP_OFF
buzzerOnDuration  = 0        // how long to stay ON
buzzerOffDuration = 0        // how long to stay OFF
buzzerCycles      = 0        // how many ON/OFF cycles remaining
buzzerCycleCount  = 0        // total cycles requested
buzzerStateTime   = 0        // timestamp of last state change
showingWarning    = FALSE
warningStartTime  = 0
WARNING_DURATION  = 1500
sc1PingSent = FALSE
p1StopSent = FALSE
```

Screen Definitions

```
SCREEN_HOME     = 0
SCREEN_PUMPS    = 1
SCREEN_LEVELS   = 2
SCREEN_TIMER    = 3
SCREEN_SOURCE   = 4
SCREEN_WIFI     = 5
SCREEN_FAULT    = 6
SCREEN_SAVER    = 7
```

Button Thresholds

```
BTN_NONE    = 1023
BTN_TOGGLE  = 800
BTN_SELECT  = 400
```

```
BTN_BACK     = 200
```

BOOT

```
ON POWER UP:
    pinMode(BUZZER, OUTPUT)
    digitalWrite(BUZZER, LOW)
    HC12.begin(9600)
    tft.init()
    tft.setRotation(1)          // landscape
    tft.fillScreen(BLACK)
    p1StopSent = FALSE

    // No boot delay -- Control has no high voltage components

    ntpSync()
    drawStaticFrame()
    sc1PingSent = FALSE

    lastActivityTime = now()
    lastHeartbeat    = now()
    currentScreen    = SCREEN_HOME
    drawHomeScreen()

    GOTO IDLE
```

NTP_SYNC

```
FUNCTION ntpSync():
    WiFi.begin(SSID, PASSWORD)
    startAttempt = now()
    WHILE WiFi.status() != CONNECTED AND (now() - startAttempt) < 10000:
        // Keep processing critical tasks while waiting for WiFi
        IF HC12.available() >= 6:
            readPacket()
        connectivityMonitor()
        processBuzzer()
        processQueue()
        wait(50ms)      // shorter wait -- more responsive
    IF WiFi.status() == CONNECTED:
        lastConnectionSuccess = TRUE
```

```
        timeClient.begin()
        ntpStart = now()
        WHILE NOT timeClient.update() AND (now() - ntpStart) < 5000:
             IF HC12.available() >= 6:
             readPacket()
             processQueue()
             wait(50ms)
        WiFi.disconnect()
        WiFi.mode(OFF)
    ELSE:
        lastConnectionSuccess = FALSE
    nextNTPSync = now() + getMillisUntil7AM()
```

WiFi is used only for NTP — connect, sync, disconnect. The internal clock runs independently afterward. `lastConnectionSuccess` is set here too so the WiFi screen shows the correct icon on boot.

IDLE (main loop)

```
LOOP:
    now = millis()

    IF now >= nextNTPSync:
    // Show sync indicator on WiFi screen icon if visible
    IF currentScreen == SCREEN_WIFI:
        drawSyncingIndicator()    // simple "syncing..." text
    ntpSync()

    IF currentScreen == SCREEN_WIFI:
        drawInnerScreen()         // redraw after sync completes

    // --- Screensaver check ---
    IF (now - lastActivityTime) >= INACTIVITY_LIMIT:
        IF NOT screensaverActive:
            showScreensaver()


 // --- Update clock every loop ---
    updateClock()

    // --- Heartbeat polling ---
    // SC1 at 1s mark, SC2 at 31s mark, reset at 31s
```

```
   elapsed = now - lastHeartbeat

    IF elapsed >= SC2_POLL_OFFSET:
       pushToQueue(ADDR_SC2, CMD_PING, 0x00, PRIORITY_NORMAL)
       lastHeartbeat = now()
       sc1PingSent = FALSE          // reset for next cycle
    ELSE IF elapsed >= SC1_POLL_OFFSET AND NOT sc1PingSent:
       pushToQueue(ADDR_SC1, CMD_PING, 0x00, PRIORITY_NORMAL)
       sc1PingSent = TRUE           // lock it until next cycle

    // --- Timer countdown update every 30 seconds ---
IF (now - lastCountdownUpdate) >= 30000:
    IF timerActive AND now >= timerEndMillis:
        // Timer expired locally but CMD_TIMER_DONE never arrived
        timerActive = FALSE
    updateTimerCountdown()
    lastCountdownUpdate = now()

    // --- Incoming packet handler ---
    IF HC12.available() >= 6:
        readPacket()

    // NEW
IF btnVal < BTN_NONE AND (now() - lastButtonTime) >= DEBOUNCE_INTERVAL:
    lastButtonTime = now()
    lastActivityTime = now()
    lightBeep()
    IF screensaverActive:
        wakeFromScreensaver()
    ELSE:
        handleInput(btnVal)

    // --- Connectivity monitor ---
    connectivityMonitor()

    // --- Auto logic ---
    evaluateAutoLogic()

    // --- Process outgoing queue ---
    processQueue()

// --- Buzzer state machine --- processBuzzer()
```

```
// --- Warning timeout ---
IF showingWarning AND (now() - warningStartTime) >= WARNING_DURATION:
    showingWarning = FALSE
    drawInnerScreen()

    REPEAT LOOP
```

Clock updates every loop but only redraws if time has changed. Heartbeat uses else-if to prevent SC1 ping firing every loop — SC1 fires at 1s mark, SC2 at 31s mark, then timer resets.

READ_PACKET

```
FUNCTION readPacket():
    FOR i = 0 to 5:
        inPacket[i] = HC12.read()

    // Validate preamble
    IF inPacket[0] != 0xAA OR inPacket[1] != 0x55:
        flush HC12 buffer
        RETURN

    // Validate destination
    IF inPacket[2] != ADDR_MASTER:
        flush HC12 buffer
        RETURN

    // Validate checksum
    IF NOT validateChecksum(inPacket):
        flush HC12 buffer
        RETURN

    processIncoming()
```

PROCESS_INCOMING

```
FUNCTION processIncoming():
    cmd = inPacket[3]
    val = inPacket[4]

    // --- SC1 level report ---
```

```
    IF cmd == CMD_REPORT_SC1:
        c1Level       = val
        sc1Connected  = TRUE
        lastSC1Heard  = now()
        pushToQueue(ADDR_SC1, CMD_ACK, CMD_REPORT_SC1, PRIORITY_LOW)
        updateLevelDisplay()
        updateSignalIcons()
        RETURN

    // --- SC2 level report ---
    IF cmd == CMD_REPORT_SC2:
        c2Level       = val
        sc2Connected  = TRUE
        lastSC2Heard  = now()
        pushToQueue(ADDR_SC2, CMD_ACK, CMD_REPORT_SC2, PRIORITY_LOW)
        updateLevelDisplay()
        updateSignalIcons()
        RETURN

    IF cmd == CMD_PUMP:
    sc2Connected = TRUE
    lastSC2Heard = now()
    SWITCH val:
        case 0:
            pumps[0].isOn = FALSE
            pumps[0].statusLabel = "OFF"

        case 1:
            pumps[0].isOn = TRUE
            pumps[0].isLocked = FALSE    // unlock confirmed here, not
before
            pumps[0].statusLabel = "ON"


        case 2:
            pumps[1].isOn = FALSE
            pumps[1].statusLabel = "OFF"
            IF timerActive:
                timerActive = FALSE
                IF currentScreen == SCREEN_TIMER:
                    updateTimerCountdown()    // redraw timer screen to
show cleared
```

```
        case 3:
            pumps[1].isOn = TRUE
            pumps[1].isLocked = FALSE    // unlock confirmed here, not
before
            pumps[1].statusLabel = "ON"
    pushToQueue(ADDR_SC2, CMD_ACK, CMD_PUMP, PRIORITY_LOW)
    IF currentScreen == SCREEN_PUMPS:
        updateLightningIndicators()
    RETURN

    IF cmd == CMD_TIMER_DONE:
        sc2Connected = TRUE
        lastSC2Heard = now()
        timerActive = FALSE
        pushToQueue(ADDR_SC2, CMD_ACK, CMD_TIMER_DONE, PRIORITY_LOW)
    IF currentScreen == SCREEN_TIMER:
        updateTimerCountdown()   // redraws UI to show timer cleared
    RETURN

    // --- Fault from SC2 ---
    IF cmd == CMD_FAULT:
        sc2Connected  = TRUE
        lastSC2Heard  = now()

        pendingFault  = val
        waitingForAmp = TRUE

        // Lock faulted pump immediately
        IF val == 1 OR val == 2:
            pumps[0].isOn        = FALSE
            pumps[0].isFault     = TRUE
            pumps[0].isLocked    = TRUE
            pumps[0].statusLabel = "FAULT"
        IF val == 3 OR val == 4:
            pumps[1].isOn        = FALSE
            pumps[1].isFault     = TRUE
            pumps[1].isLocked    = TRUE
            pumps[1].statusLabel = "FAULT"

        // Do NOT ACK yet -- waiting for AMP packet
        RETURN
```

```
    // --- Amperage report from SC2 (always follows fault) ---
    IF cmd == CMD_AMP_P1 OR cmd == CMD_AMP_P2:
        sc2Connected  = TRUE
        lastSC2Heard  = now()

        IF waitingForAmp:
          lastKnownAmps = val / 10.0
          waitingForAmp = FALSE
          activeFault = pendingFault
          pendingFault = 0
          pushToQueue(ADDR_SC2, CMD_ACK, CMD_FAULT, PRIORITY_LOW)
          pushToQueue(ADDR_SC2, CMD_ACK, cmd, PRIORITY_LOW)  // ACK the
AMP command itself
    previousScreen = currentScreen
    currentScreen = SCREEN_FAULT
    drawStaticFrame()
    drawFaultScreen()
ELSE:
    // Duplicate AMP -- ACK both so SC2 clears its queue cleanly
    pushToQueue(ADDR_SC2, CMD_ACK, CMD_FAULT, PRIORITY_LOW)
    pushToQueue(ADDR_SC2, CMD_ACK, cmd, PRIORITY_LOW)

            // Trigger fault screen
            previousScreen = currentScreen
            currentScreen  = SCREEN_FAULT
            drawStaticFrame()
            drawFaultScreen()               // includes faultBeep()
        RETURN

    // --- ACK from SC1 or SC2 ---
    IF cmd == CMD_ACK:
        handleACK(val)
        RETURN
```

Master waits for both CMD_FAULT and CMD_AMP before ACKing. This ensures amperage is always captured before the fault screen is shown. Pump is locked immediately on CMD_FAULT receipt without waiting for AMP.

EVALUATE_AUTO_LOGIC

```
FUNCTION evaluateAutoLogic():

    IF sourceActive:
        p1StopSent = FALSE           // reset flag while source is active
    IF c1Level >= C1_HIGH:
        sourceActive = FALSE
    IF pumps[0].isOn:
            pushToQueue(ADDR_SC2, CMD_PUMP, 0, PRIORITY_IMPORTANT)
            p1StopSent = TRUE    // already sent, don't send again below
    ELSE IF pumps[0].isOn == FALSE AND NOT pumps[0].isLocked:
        pushToQueue(ADDR_SC2, CMD_PUMP, 1, PRIORITY_IMPORTANT)


    IF NOT sourceActive AND pumps[0].isOn AND NOT p1StopSent:
       pushToQueue(ADDR_SC2, CMD_PUMP, 0, PRIORITY_IMPORTANT)
       p1StopSent = TRUE

    // --- P2 logic (reservoir → overhead tank) ---
    IF pumps[1].isLocked: RETURN           // never auto-start locked pump

    IF c2Level < C1_LOW AND c1Level > C1_LOW AND NOT pumps[1].isOn:
        pushToQueue(ADDR_SC2, CMD_PUMP, 3, PRIORITY_IMPORTANT)

    IF c2Level >= C2_FULL AND pumps[1].isOn:
        pushToQueue(ADDR_SC2, CMD_PUMP, 2, PRIORITY_IMPORTANT)

    IF c1Level <= C1_LOW AND pumps[1].isOn:
        pushToQueue(ADDR_SC2, CMD_PUMP, 2, PRIORITY_IMPORTANT)
```

P1 is driven by source flag and C1 level. P2 is driven by C2 demand and C1 availability. isLocked blocks auto logic from restarting a faulted pump — only manual intervention from the pump screen can unlock.

CONNECTIVITY_MONITOR

```
FUNCTION connectivityMonitor():
    changed = FALSE

    IF sc1Connected AND (now() - lastSC1Heard) > SC_TIMEOUT:
        sc1Connected = FALSE
        changed      = TRUE

    IF sc2Connected AND (now() - lastSC2Heard) > SC_TIMEOUT:
    sc2Connected = FALSE
    changed = TRUE
    // Update UI to reflect SC2 has gone silent
    // SC2 already self-froze P2 at 120s -- no point sending commands
    // that won't arrive. Just correct local state.
    IF pumps[0].isOn OR pumps[1].isOn:
        pumps[0].isOn = FALSE
        pumps[1].isOn = FALSE
        pumps[0].statusLabel = "OFF"
        pumps[1].statusLabel = "OFF"
        timerActive = FALSE      // mirror SC2's own timer cancellation

    IF changed:
        updateSignalIcons()
```

3 minute timeout. Master sends pump OFF commands when SC2 goes silent even though they are unlikely to arrive — this is a safety measure to ensure SC2 receives them if connectivity recovers quickly.

HANDLE_INPUT

```
FUNCTION handleInput(btnVal):
    IF btnVal < BTN_BACK:
        onBackPressed()
    ELSE IF btnVal < BTN_SELECT:
        onSelectPressed()
    ELSE IF btnVal < BTN_TOGGLE:
        onTogglePressed()
```

ON_TOGGLE_PRESSED

```
FUNCTION onTogglePressed():
    // Fault screen -- ignore all toggle
    IF currentScreen == SCREEN_FAULT: RETURN

    IF currentScreen == SCREEN_HOME:
        menuIndex = (menuIndex + 1) % maxMenuItems
        updateMenuHighlight()
        RETURN

    IF currentScreen == SCREEN_PUMPS:
        IF NOT selectingAction:
            selectedPump = (selectedPump + 1) % 2
        ELSE:
            pumpActionIndex = (pumpActionIndex + 1) % 2
        updatePumpSelection()
        RETURN

    IF currentScreen == SCREEN_TIMER:
        IF NOT timerActive:
            timerIndex = (timerIndex + 1) % 9
            updateTimerPreset()
        // if timer active, ignore silently
        RETURN

    IF currentScreen == SCREEN_SOURCE:
        isDisabled = timerActive OR (c2Level >= C2_FULL)
        IF isDisabled: RETURN
        sourceToggleIndex = (sourceToggleIndex + 1) % 2
        updateSourceDisplay()
        RETURN

    // SCREEN_WIFI and SCREEN_LEVELS -- nothing to toggle
```

ON_SELECT_PRESSED

```
FUNCTION onSelectPressed():
    // Fault screen -- clear fault
    IF currentScreen == SCREEN_FAULT:
        clearFault()
        RETURN

    IF currentScreen == SCREEN_HOME:
```

```
        previousScreen = SCREEN_HOME
        SWITCH menuIndex:
            case 0: currentScreen = SCREEN_PUMPS
            case 1: currentScreen = SCREEN_LEVELS
            case 2: currentScreen = SCREEN_TIMER
            case 3: currentScreen = SCREEN_SOURCE
            case 4: currentScreen = SCREEN_WIFI
        drawInnerScreen()
        RETURN

    IF currentScreen == SCREEN_PUMPS:
        IF NOT selectingAction:
            selectingAction = TRUE
            pumpActionIndex = 0             // default to ON
            updatePumpSelection()
        ELSE:
            selectPumpAction()
        RETURN

    IF currentScreen == SCREEN_TIMER:
        IF timerActive: RETURN              // ignore silently
        sendTimerCommand(timerPresets[timerIndex])
        timerActive    = TRUE
        timerEndMillis = now() + (timerPresets[timerIndex] × 60000)
        updateTimerCross()
        updateTimerCountdown()
        RETURN

    IF currentScreen == SCREEN_SOURCE:
        isDisabled = timerActive OR (c2Level >= C2_FULL)
        IF isDisabled: RETURN
        sourceActive = (sourceToggleIndex == 0)
        updateSourceDisplay()
        RETURN

    IF currentScreen == SCREEN_WIFI:
        IF NOT runningDiagnostic:
            runWifiDiagnostic()
        RETURN
```

ON_BACK_PRESSED

```
FUNCTION onBackPressed():
    // Fault screen -- cannot back out
    IF currentScreen == SCREEN_FAULT: RETURN

    // WiFi screen -- cannot back during diagnostic
    IF currentScreen == SCREEN_WIFI AND runningDiagnostic: RETURN

    // Pump screen -- if in action selection, go back to pump selection
first
    IF currentScreen == SCREEN_PUMPS AND selectingAction:
        selectingAction = FALSE
        updatePumpSelection()
        RETURN

    // All other screens -- go home
    IF currentScreen != SCREEN_HOME:
        currentScreen = SCREEN_HOME
        menuIndex     = 0
        drawInnerScreen()
```

SELECT_PUMP_ACTION

```
FUNCTION selectPumpAction():
    // Determine WACP val
    IF selectedPump == 0:
        val = (pumpActionIndex == 0) ? 1 : 0    // Overhead ON=1, OFF=0
    ELSE:
        val = (pumpActionIndex == 0) ? 3 : 2    // Basement ON=3, OFF=2

    // Check C2 fullness before Basement pump ON
    IF selectedPump == 1 AND pumpActionIndex == 0:
        IF c2Level >= C2_FULL:
            showTankFullWarning()
            RETURN

    // Send command
    pushToQueue(ADDR_SC2, CMD_PUMP, val, PRIORITY_IMPORTANT)

    // For OFF commands, update immediately -- no confirmation needed
    IF pumpActionIndex == 1:
        pumps[selectedPump].isOn = FALSE
        pumps[selectedPump].statusLabel = "OFF"
```

```
        IF pumps[selectedPump].isLocked:
            pumps[selectedPump].statusLabel = "LOCKED"

    // For ON commands, show PENDING -- wait for SC2 to confirm
    IF pumpActionIndex == 0:
        pumps[selectedPump].statusLabel = "PENDING"
        // isLocked and isOn stay as-is until SC2 confirms

    updateLightningIndicators()
    selectingAction = FALSE
    updatePumpSelection()
```

`isLocked` is cleared on manual ON command — this is the only way to unlock a pump after a fault. Users physically pressing ON confirm they have inspected the pump.

CLEAR_FAULT

```
FUNCTION clearFault():
    // Lock the faulted pump -- auto logic blocked until manual unlock
    IF activeFault == 1 OR activeFault == 2:
        pumps[0].isLocked     = TRUE
        pumps[0].isFault      = FALSE
        pumps[0].statusLabel  = "LOCKED"

    IF activeFault == 3 OR activeFault == 4:
        pumps[1].isLocked     = TRUE
        pumps[1].isFault      = FALSE
        pumps[1].statusLabel  = "LOCKED"

    activeFault   = 0
    currentScreen = previousScreen
    drawStaticFrame()
    drawInnerScreen()
```

PROCESS_QUEUE

```
FUNCTION processQueue():
    IF queueCount == 0: RETURN
    qp = packetQueue[0]
    IF qp.waitingAck:
        IF (now() - lastSendTime) >= BACKOFF_MS:
```

```
            qp.retries++
            IF qp.retries >= MAX_RETRIES:
                removeFromQueue(0)
                RETURN
            IF NOT HC12.available():    // only retry if channel is clear
                HC12.write(qp.data, 6)
                lastSendTime = now()
            RETURN
    // Send packet only if channel is clear
    IF NOT HC12.available():
        HC12.write(qp.data, 6)
        lastSendTime = now()
        qp.waitingAck = TRUE
```

HANDLE_ACK

```
FUNCTION handleACK(ackedCmd):
    srcAddr = inPacket[2]    // who sent this ACK
    FOR i = 0 to queueCount:
        IF packetQueue[i].data[3] == ackedCmd AND
           packetQueue[i].data[2] == srcAddr:
            removeFromQueue(i)
            RETURN
```

PUSH_TO_QUEUE

```
FUNCTION pushToQueue(dest, cmd, val, priority):
    IF queueCount >= QUEUE_SIZE:
        lowestIdx  = index of packet with highest priority number
        lowestPrio = its priority value

        IF priority < lowestPrio:
            removeFromQueue(lowestIdx)  // evict lowest to make room
        ELSE:
            RETURN                      // new packet not important enough

    // Build packet
    qp.data       = [0xAA, 0x55, dest, cmd, val, dest XOR cmd XOR val]
    qp.priority   = priority
    qp.retries    = 0
    qp.waitingAck = FALSE
```

```
    // Insert at sorted position
    Find insertIdx where priority < packetQueue[insertIdx].priority
    Shift packets down from insertIdx
    packetQueue[insertIdx] = qp
    queueCount++
```

SEND_TIMER_COMMAND

```
FUNCTION sendTimerCommand(minutes):
    pushToQueue(ADDR_SC2, CMD_TIMER, minutes, PRIORITY_IMPORTANT)
```

HELPERS

```
FUNCTION showTankFullWarning():
    tft.fillRect(16, 54, 288, 138, TFT_BLACK)
    tft.setTextColor(0xFFFF)
    tft.setTextSize(1)
    tft.drawString("TANK FULL!", 120, 115)
    showingWarning   = TRUE
    warningStartTime = now()
```